# AssistVision: Before-and-After Plate Weighing and Scanning for Meal-Intake Monitoring in Older Adults

Dr. Usama Abdur Rahman, M.Tech., Ph.D.
Dept. of Computer Science and Engineering
with Specialization in Artificial Intelligence
Sathyabama Institute of Science and Technology
Chennai, India

Yasir, K. M. Mohammed Mohsin
Dept. of Computer Science and Engineering
with Specialization in Artificial Intelligence
Sathyabama Institute of Science and Technology
Chennai, India

***Abstract*—Many older adults eat less than they are served, and low meal intake is an independent risk factor for death in hospital, yet intake is usually judged by eye. AssistVision is a rig costing at most INR 4,220 (ESP32-S3 camera, stepper turntable, HX711 load cell, printed ArUco mat) that weighs and scans the plate when a meal is served and again when it is cleared; the mass difference gives grams eaten. A companion app keeps the meal log, alerts family after a run of low-intake meals and prints a weekly report for the doctor. To choose the vision pipeline we test eleven monocular depth models on 507 Nutrition5k dishes and dense multi-view stereo on 22 synthetic plates with exact volume, plus 12 before/after meal pairs. From an overhead view, deployable table-only anchoring of relative models fails (median volume error $\geq$88.4 %), because a fronto-parallel plane cannot fix both scale and shift. Metric models used as-is are off by 8–54$\times$ in volume; fitting their single scale to the table makes ZoeDepth-NYU the best deployable monocular option, at 44.1 % median error. On the 16 textured-food scenes, dense 24-view MVS with marker scale reaches 4.6 % [2.0, 6.1] median absolute error against exact mesh volume, while sparse SfM alone misses 91.3 % and textureless food on a white plate fails (45.1 %). Accuracy collapses below 12 views and under heavy JPEG compression, but not at half resolution. Even sensor-depth volume predicts weighed mass worse than the dataset mean (68.2 vs 52.0 % median error), which is why the rig weighs the plate. No result in this paper comes from the physical rig. On partly eaten plates, the before/after volume difference matches the exact eaten volume to a median 2.8 % [2.0, 7.4] (11/12 within 10 %). The app and alert rule are an engineering prototype; no clinical or user study has been run.***

## I. Introduction

Older adults who eat too little lose weight and muscle and recover more slowly from illness. In the NutritionDay survey of 16,290 hospitalised adults, more than half did not finish the meal they were given, and eating only about a quarter of it carried an adjusted hazard ratio of 2.10 for death in hospital within 30 days (3.02 for eating nothing) [1]. The ESPEN geriatric guideline asks that all older persons be screened for malnutrition routinely [2]. Screening and follow-up need a number for how much was eaten. On wards that number usually comes from a nurse's visual estimate of the leftovers; checked against weighing, such estimates hold up on average across a group but are much less reliable for a single patient [3], [4]. At home there is often no record at all, and children living in another city learn of a decline only once it shows as weight loss.

Weighing the plate before and after the meal is the reference method in those validation studies [4]. It is accurate but tedious, so it is rarely done as routine. We make it automatic and send the result to the people who can act on it: AssistVision weighs and photographs the plate on a turntable at serving and at clearing, and its companion app shows grams eaten to family members and the doctor.

The camera half of the rig raises a measurement problem. Perspective projection discards absolute scale: a small plate near the camera and a large plate far away give the same pixels. Metric scale has to come from outside the image: from a reference object, a depth sensor, camera motion with known geometry, or a learned prior. We measure what each choice costs, and whether before/after volume differences are accurate enough to back up the scale.

Our contributions are:

1) A meal-intake monitoring system for older adults (Sec. IV): before/after weighing and scanning on one rig, intake arithmetic that flags impossible readings, and a companion app with family and clinician views (meal log, two-week trend, low-intake alert, weekly report).
2) An open hardware capture rig (Sec. V) that produces multi-view images, metric scale and plate mass in one button press, with firmware, host software, wiring and a bench-test protocol.
3) A benchmark of monocular depth models for food on real RGB-D data (Nutrition5k) that keeps deployable and oracle calibration apart.
4) A controlled synthetic study with exact mesh volumes: dense MVS against ground truth, ablations of the factors the rig designer controls (views, resolution, compression, marker accuracy, serveware), and partly eaten plates (E5).
5) A record of design decisions and rejected alternatives (Sec. III), each tied to a measurement or a stated assumption.

## II. Related Work

**Meal intake in care settings.** Hospitals measure intake by weighing plate waste (the reference) or, in routine practice, by visual estimation on quarter or percentage scales. A systematic review of 14 validation studies found visual estimation valid at group level with limited support for individual patients; only two of the studies used modern technology [3]. In a community hospital, visual estimates by nursing assistants differed significantly from weighed intake, with Bland–Altman limits of $-121$ to $+147$ kcal per tray [4]. Our rig automates the weighed reference. **Food volume.** Nutrition5k [5] provides overhead RGB-D video and weighed dishes; its depth-based volume baseline uses a constant per-pixel footprint at the capture-plane distance. Multi-view food reconstruction has used structure-from-motion with reference cards and dense stereo [6], [7]. **Monocular depth.** Relative depth models (MiDaS [8], Depth Anything [9], [10]) predict depth up to an unknown affine transform of inverse depth; metric models (ZoeDepth [11], Depth Pro [12], metric-finetuned Depth Anything V2) predict absolute depth from priors learned on room- and street-scale scenes. **Reconstruction.** We use COLMAP [13] for SfM and OpenMVS for dense MVS.

## III. Design Decisions and Rejected Alternatives

Table I summarises each decision; the evidence column points to the experiment that supports it. Decisions marked *assumption* rest on datasheets or cost quotes, not on our measurements.

## IV. Meal-Intake Monitoring for Older Adults

**Protocol.** The empty plate is tared. At serving the rig records the food mass $m_s$ and a 24-view scan (volume $V_s$); when the plate is cleared it records $m_l$ and $V_l$ (Fig. 1; `rig_client.py meal --phase served|cleared`). Eaten mass and fraction are

$$m_e = m_s - \min\big(\max(m_l, 0),\, m_s\big), \qquad f = m_e/m_s. \quad (1)$$

A served reading under 2 g is treated as an empty plate. A leftover heavier than the serving by more than 2 g (food added, plate swapped, a spoon left on the plate) is stored with a flag and excluded from alerts instead of being silently clipped. The volume difference $V_s - V_l$ is stored next to $m_e$ as a cross-check; E5 measures how accurate it is.

**Why mass, not image volume, gives the grams.** Converting volume to mass needs a density for every food, and on real mixed dishes sensor-derived volume barely tracks weighed mass (E2). The load cell measures mass directly, so the reported intake never depends on a density table.

**Alerts and reports.** A meal is *low* if $f < \tau$; an alert is raised when the last $k$ scored meals are all low, and only once per run. The defaults $\tau = 50\,\%$ and $k = 3$ are prototype values, not clinical cut-offs; the clinician sets both per person. The weekly report lists, per day, the number of meals, grams served, grams eaten and the eaten share, plus the count of low meals. It reports food mass, not energy or nutrients.

**App.** The companion app (Flask and SQLite, Fig. 2) has one page per older adult with the alert banner, weekly figures, two-week chart, recent meals and the circle of care (family members and doctor). The rig posts each cleared plate to `/api/meals`; meals can also be entered by hand. It stores masses, times and meal names, not images, which stay on the host for reconstruction. In this prototype alerts are recorded in the app but not sent by SMS or e-mail. Unit tests cover the intake arithmetic, the alert rule (including firing once per run) and the rig-to-app path.

## V. Hardware

The rig (Figs. 3–5) uses an ESP32-S3 camera board (OV2640, up to $1600 \times 1200$), a 28BYJ-48 stepper through a ULN2003 driver (4076 half-steps per revolution, 0.088° per step), and a 5 kg bar load cell read by an HX711 at 10 Hz. The firmware exposes `/capture`, `/rotate`, `/weight`, `/tare`, `/calibrate` and camera-setting endpoints. Exposure, gain and white balance are locked after convergence so that all views of a scan share photometry, which MVS assumes. Stepper coils are released while weighing to avoid heating and vibration of the load cell. Tare and the counts-per-gram calibration persist in flash. The firmware compiles for the ESP32-S3 target with the Arduino core 3.3.12; a fake-firmware test exercises every client endpoint, retry on a dropped frame, and calibration arithmetic.

**Physical validation (pending).** Load-cell linearity, repeatability and drift, turntable angular error, scan duration and transfer latency are specified as scripted bench tests (`rig_client.py bench-`$\star$) whose results are not yet measured, because the rig has not yet been assembled. We report no physical number we have not measured.

## VI. Method

### A. Shared volume integral

All methods produce a 3-D point set $P$ in metres and a support plane $(\mathbf{n}, d)$ oriented towards the camera. Each point gets a height $h = \mathbf{n}^\top \mathbf{p} + d$ and plane coordinates in an orthonormal basis; points are binned into square cells of side $c$ and each cell keeps the median height. Empty cells inside the footprint are filled by nearest neighbour, negative heights are clipped, and

$$V = c^2 \sum_{\text{cells}} \max(h, 0). \quad (2)$$

Because monocular, multi-view and sensor depth all share this integral, differences in volume reflect differences in geometry, not in post-processing.

### B. Metric scale from the marker mat

Marker corners are detected in every view and triangulated with the SfM cameras; the scale $s$ is the ratio of the known side to the median triangulated side. A relative error $\epsilon$ in $s$ produces a volume error of $(1+\epsilon)^3 - 1 \approx 3\epsilon$, so a 1 % scale error costs about 3 % in volume; Sec. VII-C measures this directly.

TABLE I
DESIGN DECISIONS, ALTERNATIVES CONSIDERED, AND THE EVIDENCE BEHIND EACH CHOICE.

| Decision | Chosen | Rejected alternatives | Why / evidence |
|---|---|---|---|
| Scale source | Printed ArUco mat, multi-view triangulation | Credit-card reference in one image; metric monocular depth; depth sensor | Volume scales with the *cube* of length error (Sec. VI-B); metric networks are off by large factors at plate distance (Table II); a D435-class sensor costs $\sim$10$\times$ the camera (assumption). |
| Geometry | Dense MVS (COLMAP + OpenMVS) | Sparse SfM points only; single-view depth | Sparse points under-cover low-texture food (Sec. VII-C); single view cannot see the far side of a mound even with perfect depth. |
| Capture motion | Fixed camera, motorised turntable | Hand-held phone orbit; multi-camera ring | Repeatable baselines and known angles; one camera keeps cost and synchronisation trivial; the angle prior is checked against SfM. |
| Camera tilt | 45° elevation | Straight overhead (Nutrition5k) | Overhead view makes the support plane fronto-parallel: the relative-depth anchor fit becomes rank-deficient (condition number $10^{14}$, Sec. VI-C). |
| Controller | ESP32-S3 + OV2640 | Raspberry Pi + camera; smartphone app | Pi adds cost, boot time and an OS to maintain; phones vary per user and cannot drive the turntable or scale. Processing is host-side, so the MCU only needs camera, Wi-Fi and GPIO (assumption: cost table in hardware README). |
| Ground truth | 5 kg bar load cell + HX711 under the plate | External kitchen scale; no mass at all | In-rig weighing removes a manual step and lets every scan carry its own reference; tare-before-serve gives food-only mass. |
| Volume integral | Height field over the fitted support plane | Constant per-pixel footprint (Nutrition5k); voxel carving; mesh + watertight closure | Exact on tilted planes in our unit tests (0.09 % error) where the constant-footprint formula is not; shared across all methods so comparisons measure geometry, not bookkeeping. |
| Intake measure | Load-cell difference, served minus left | Staff visual estimate; image-volume difference $\times$ density | Weighing is the reference in validation studies [4]; visual estimates are weak for individuals [3]; volume needs density and tracks mass poorly (E2). |
| Alert rule | $k$ low meals in a row, threshold set by the clinician | Daily-total threshold; fixed population cut-off | A run of low meals is visible within a day; thresholds are prototype defaults, not validated cut-offs. |
| Mono calibration | Report raw / anchor / oracle separately | Report only oracle-aligned numbers | Oracle alignment uses the ground truth being evaluated; it is an upper bound, not a deployable result. |

### C. Monocular protocols

**Raw**: the model's metric output is used as-is. **Anchor**: the prediction is calibrated using only the support plane (the table in Nutrition5k, the marker mat in our rig), never the food; metric outputs get one scale, relative outputs an affine map of inverse depth. **Oracle**: the same maps fitted to ground-truth depth over the whole dish region—an upper bound that is not available at deployment. For relative models we record the condition number of the anchor fit; a fronto-parallel plane has nearly constant inverse depth, so scale and shift are not separately identifiable.

## VII. EXPERIMENTS

### A. E1: Monocular depth on Nutrition5k

**Relative models: anchoring on the table fails from overhead.** The five relative models have a median volume error of 88.4–99.4 % under the deployable anchor protocol (Table II), even though their predicted table is flat to 0.6–0.7 mm RMS: the scale-and-shift fit reproduces the table and collapses the food relief. The fit is badly conditioned on most dishes (Fig. 6), as expected when the only reference is a plane almost perpendicular to the optical axis. This is the quantitative reason the rig views the plate at $45^\circ$ and uses the marker mat and multiple views.

**Metric models are not metric at plate distance, but one scale helps.** Used as-is, all five metric models are far off: AbsRel 1.33–2.76 and median volume errors of 759–801 % (DA-V2 metric indoor) and 3,539–5,328 % (ZoeDepth). ZoeDepth places the table at about 1.2 m instead of 0.35 m, a distance well outside the room-scale scenes it was trained on. Their anchor fits only one scale to the table, which is well conditioned even from overhead. ZoeDepth-NYU is then the best deployable model: 44.1 % [39.6, 50.5] median volume error, 24 % of dishes within 20 %, ahead of the next model

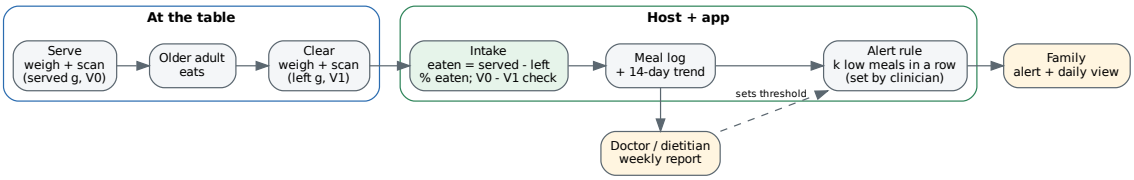



Fig. 1. Meal-intake loop. The rig records served and leftover mass and volume; the host computes intake; family members get alerts and a daily view; the doctor gets a weekly report and sets the alert threshold.

TABLE II
E1 (NUTRITION5K, $n$ DISHES PER MODEL AS SHOWN). DEPTH ERROR OVER THE DISH REGION AND VOLUME ERROR RELATIVE TO SENSOR-DEPTH VOLUME, PER MODEL AND CALIBRATION PROTOCOL. BRACKETS: 95% BOOTSTRAP CI (10 000 RESAMPLES). LATENCY IS CPU TIME PER IMAGE ON 8-VCPU CLOUD VMS (VM EVIDENCE; SEE TEXT).

| Model | Prot. | AbsRel | $h$ MAE (mm) | Vol. APE med. (%) | $\leq$20% (%) | APE, sens. plane (%) | table RMS (mm) | s/img |
|---|---|---|---|---|---|---|---|---|
| DepthPro | raw | 0.201 | 19.0 | 97.9 [79.4, 100.0] | 12 | 100.0 | 9.2 | 40.08 |
| DA-V2-Metric-Indoor-L | raw | 1.546 | 58.3 | 759.3 [627.7, 860.3] | 2 | 100.0 | 26.7 | 3.80 |
| DA-V2-Metric-Indoor-B | raw | 2.028 | 118.2 | 772.2 [356.3, 1090.1] | 2 | 100.0 | 51.5 | 2.18 |
| DA-V2-Metric-Indoor-S | raw | 1.334 | 55.8 | 801.3 [645.5, 942.5] | 3 | 100.0 | 25.4 | 1.70 |
| ZoeDepth-NYU-KITTI | raw | 2.759 | 79.0 | 3539.3 [3267.7, 3906.8] | 0 | 100.0 | 34.9 | 2.76 |
| ZoeDepth-NYU | raw | 2.613 | 86.2 | 5327.9 [5074.6, 5574.9] | 0 | 100.0 | 34.1 | 2.64 |
| ZoeDepth-NYU | anchor | 0.049 | 18.5 | 44.1 [39.6, 50.5] | 24 | 49.4 | 9.4 | 2.64 |
| DepthPro | anchor | 0.037 | 14.7 | 47.7 [41.7, 54.9] | 24 | 47.4 | 7.9 | 40.08 |
| ZoeDepth-NYU-KITTI | anchor | 0.050 | 21.4 | 58.6 [54.3, 62.8] | 18 | 53.2 | 9.3 | 2.76 |
| DA-V2-Metric-Indoor-S | anchor | 0.063 | 24.4 | 73.5 [67.3, 80.3] | 12 | 70.7 | 11.1 | 1.70 |
| DA-V2-Metric-Indoor-L | anchor | 0.057 | 25.8 | 82.7 [77.7, 86.4] | 12 | 75.6 | 10.5 | 3.80 |
| DA-V2-L | anchor | 0.035 | 12.5 | 88.4 [86.2, 90.1] | 6 | 76.3 | 0.7 | 3.94 |
| DA-V2-B | anchor | 0.038 | 13.4 | 92.7 [90.1, 95.4] | 4 | 84.9 | 0.6 | 2.89 |
| DA-V1-S | anchor | 0.039 | 13.6 | 94.8 [92.9, 96.0] | 3 | 86.1 | 0.6 | 1.64 |
| DA-V2-S | anchor | 0.038 | 13.4 | 96.0 [93.4, 97.8] | 3 | 86.9 | 0.6 | 1.85 |
| MiDaS-DPT-Hybrid | anchor | 0.045 | 15.8 | 99.4 [98.9, 99.8] | 1 | 93.7 | 0.7 | 1.39 |
| DA-V2-Metric-Indoor-B | anchor | 0.125 | 45.8 | 99.7 [98.7, 99.9] | 7 | 99.6 | 18.3 | 2.18 |
| DA-V2-L | oracle | 0.013 | 11.2 | 25.9 [23.7, 30.2] | 38 | 9.6 | 3.0 | 3.94 |
| DA-V2-B | oracle | 0.015 | 11.8 | 32.1 [28.5, 35.7] | 29 | 13.0 | 2.7 | 2.89 |
| DA-V2-S | oracle | 0.016 | 12.0 | 37.4 [33.0, 41.9] | 26 | 14.1 | 2.6 | 1.85 |
| ZoeDepth-NYU | oracle | 0.036 | 20.0 | 45.1 [40.7, 50.8] | 23 | 33.3 | 9.5 | 2.64 |
| DA-V1-S | oracle | 0.017 | 10.3 | 47.6 [42.9, 51.8] | 17 | 17.2 | 2.4 | 1.64 |
| DepthPro | oracle | 0.028 | 15.1 | 49.5 [43.5, 57.7] | 23 | 30.5 | 7.9 | 40.08 |
| ZoeDepth-NYU-KITTI | oracle | 0.037 | 22.5 | 60.9 [55.7, 65.2] | 18 | 31.6 | 9.3 | 2.76 |
| DA-V2-Metric-Indoor-S | oracle | 0.042 | 25.3 | 75.2 [67.1, 80.7] | 12 | 44.3 | 10.9 | 1.70 |
| DA-V2-Metric-Indoor-L | oracle | 0.037 | 25.9 | 85.5 [80.8, 89.2] | 11 | 41.6 | 10.4 | 3.80 |
| MiDaS-DPT-Hybrid | oracle | 0.022 | 11.4 | 86.9 [82.7, 89.0] | 3 | 33.8 | 1.1 | 1.39 |
| DA-V2-Metric-Indoor-B | oracle | 0.068 | 42.7 | 99.8 [99.2, 100.0] | 6 | 67.3 | 17.5 | 2.18 |

(DA-V2-Metric-Indoor-S, 73.5 %; Holm-corrected Wilcoxon $p = 3.6 \times 10^{-9}$). That is still an order of magnitude worse than multi-view reconstruction (Sec. VII-C). Its predicted table is not flat (9.4 mm RMS), and the KITTI-mixed variant is worse (58.6 %). Model size does not order the DA-V2 metric models: B is the worst of the three under every protocol.

**Depth Pro is the exception, but no better once anchored.** Depth Pro was run on a seeded 200-dish subset because it takes 40 s per image on the VM CPU. Used as-is it is the only metric model with roughly the right scale (AbsRel 0.20), yet it still overestimates volume (median signed error +82 %, absolute 97.9 %). Anchored, it reaches 47.7 % [41.7, 54.9], indistinguishable from ZoeDepth-NYU on the same dishes (45.3 %; Wilcoxon $p = 0.8$, $n = 198$) at about 15$\times$ the latency. Its oracle fit (49.5 %) is no better than its anchor and far behind DA-V2-L ($p = 9.2 \times 10^{-11}$).

**Depth metrics do not predict volume.** After alignment AbsRel is 0.013–0.125 for every model, yet volume errors span 10–100 %. Food relief (1–3 cm) is a small fraction of the 36 cm camera distance, so a model can score well on depth and still lose most of the volume. We therefore rank models on volume, not on AbsRel.

**Oracle upper bound.** With an oracle fit DA-V2-L is best of all eleven models: 25.9 % [23.7, 30.2] median volume error with its own support plane and 9.6 % with the sensor's plane. Within Depth Anything V2 (relative) the error falls with encoder size, from 37.4 % (S) to 32.1 % (B) to 25.9 % (L); L is separated from B with Holm-corrected $p = 3.1 \times 10^{-19}$. The gap between the two plane choices shows that fitting the support plane in predicted depth causes more than half

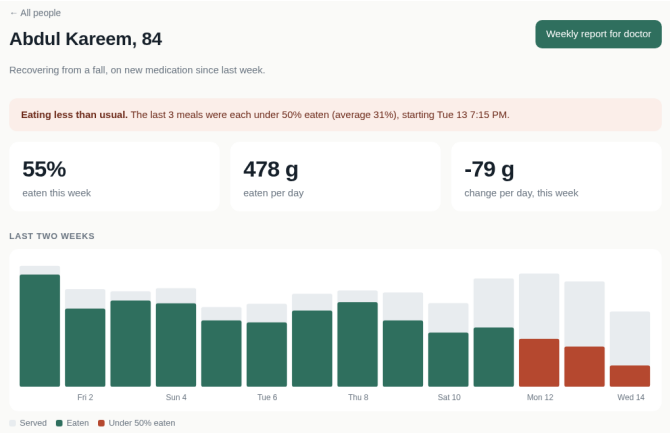



Fig. 2. Companion app, resident page (fictional demo data): alert banner, weekly share of served food eaten, grams per day, and the two-week trend of served (grey) versus eaten (green; red when under the threshold).

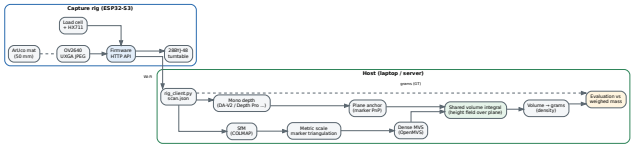



Fig. 3. System architecture. The rig exposes an HTTP API; the host drives the scan loop, so dropped frames are retried rather than corrupting a scan.

of the error; in the rig the marker mat provides this plane independently of the network. The oracle needs food ground truth and is not deployable.

**Coverage and failures.** 498 of the 503 usable dishes were evaluated for every model except Depth Pro (198 of its 200-dish subset). The remaining five have no visible table (degenerate plane fit) and are counted as failures, not dropped. Latency is 1.4–3.9 s per image on 8-vCPU cloud VMs (VM evidence), and 40 s for Depth Pro: DA-V1-S, DA-V2-S/B and MiDaS ran on a machine shared with other jobs, so their times are upper bounds, and the other models ran alone on identical VMs.

### *B. E2: Sensor-referenced volume and weighed mass*

Nutrition5k has no food-volume label, so the volume target in E1/E2 is the volume integrated from RealSense depth over the dish region with the same height-field integral (Eq. 2) applied to each model's prediction. The error therefore isolates the depth model: a model with zero error here reproduces the sensor, whatever the sensor's own error on food is. Even sensor-depth volume with a cross-validated density predicts mass worse than ignoring the image and predicting the mean (Table III). Sensor volume and weighed mass are only weakly correlated (Pearson $r = 0.19$, Spearman $\rho = 0.17$, $n = 503$). Mixed dishes differ in density, and the dish region includes the plate rim. A single image-derived volume therefore cannot stand in for grams on real dishes, and this is the quantitative case for weighing the plate in the rig. Mass counts $n$ vary

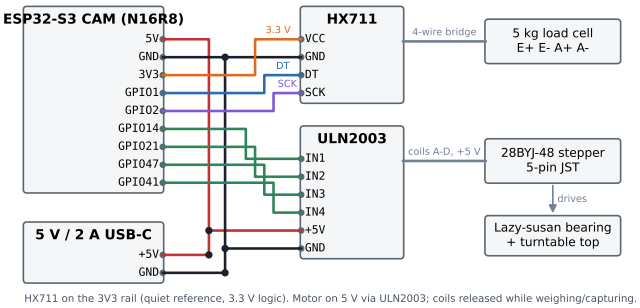



Fig. 4. Wiring of the ESP32-S3, HX711 load-cell amplifier and ULN2003 stepper driver.

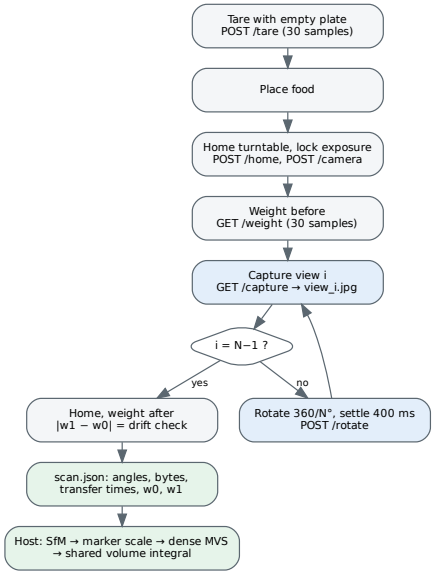



Fig. 5. Scan protocol implemented by `rig_client.py scan`. Weight is read before and after the turntable moves; their difference is a per-scan drift/vibration check.

because dishes with a non-positive predicted volume are excluded from the density fit.

### *C. E3: Multi-view versus monocular on exact synthetic volumes*

**Scenes.** We specify twenty-two Blender (Cycles) scenes with watertight food meshes whose volume is computed exactly from the mesh: eight food configurations (single rice mound, rice + dal, two idli, curry heap, lumpy rice, a three-item thali, a 40 mL small portion and a tall mound) on ceramic and polished steel plates (16 scenes), four textureless-food variants of the first four configurations on ceramic, and two configurations on banana leaf. All 22 were rendered and reconstructed. Each scene is rendered at $1600 \times 1200$, $60°$ horizontal FoV, $45°$ elevation and 0.40 m distance, with the plate rotating on a turntable inside a printed 50 mm ArUco

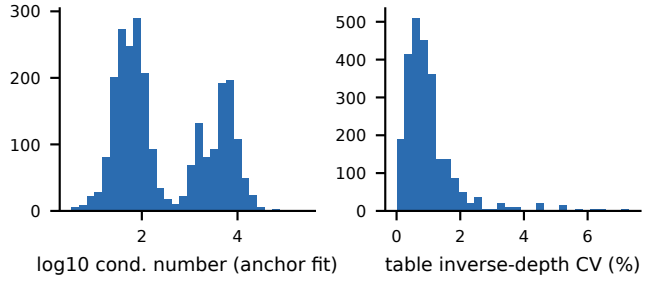



Fig. 6. Anchor-fit conditioning on Nutrition5k (relative models): condition number of the affine fit (left) and coefficient of variation of table inverse depth (right). A low CV means the table alone cannot separate scale from shift.

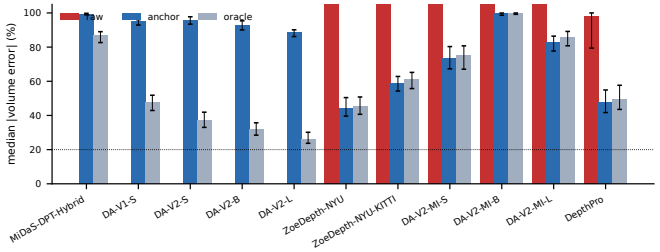



Fig. 7. Per-dish absolute volume error versus sensor depth, by model and protocol (Nutrition5k).

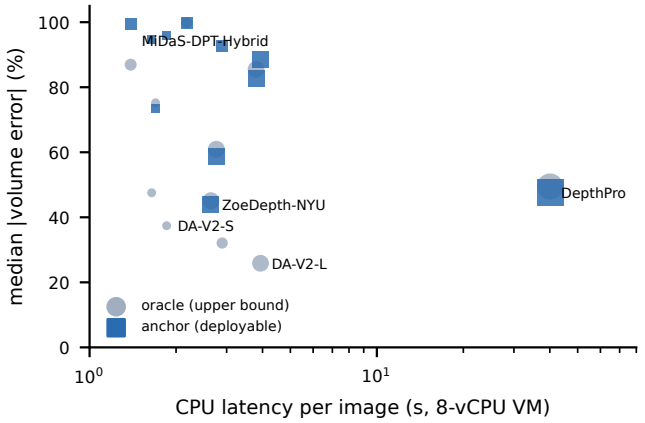



Fig. 8. Accuracy–latency trade-off (anchor and oracle protocols; CPU latency on 8-vCPU cloud VMs).

mat, i.e. the rig geometry. View 0 also stores a metric depth map for a "perfect single-view depth" baseline.

**Methods.** (i) Dense MVS: COLMAP SfM, marker-triangulated scale, OpenMVS densification, plate-noise-calibrated footprint threshold, shared integral. (ii) Sparse SfM points only, with the same scale and integral. The monocular and perfect-single-view baselines are implemented, but their runs had not finished at the time of writing and are not reported.

**Footprint rule.** Dense MVS leaves millimetre-level noise on the plate surface. A cell counts as food if its height exceeds $\mu + 2\sigma$ of the plate annulus heights. This rule was fixed on `rice_mound__ceramic` before the other scenes were run, and those scenes are held out. A hysteresis variant (seed at $2\sigma$, grow connected cells down to $0.5\sigma$) was added *after* the curry failure below, and is reported as post hoc.

Table IV gives the results. With the pre-registered footprint rule, dense MVS has a median absolute error of 4.6 % [2.0, 6.1] on the 16 textured-food scenes, and 13 of 16 are within 10 %. Steel plates (3.0 %, 8 of 8 within 10 %) do better than ceramic (5.9 %, 5 of 8). Banana leaf gives $-1.9$ and $+7.8$ %. The textureless white ceramic gives plate-surface noise of 1.9–7.4 mm, against 0.9–1.2 mm on the textured steel. The three textured-food misses are all on ceramic: the curry heap ($-26.0$ %, below), the 38 mL small portion ($-30.1$ %; 4.5 mm plate noise is a large share of an 18 mm-high portion) and the tall mound ($+16.8$ %).

**Textureless food fails.** With the food texture removed, dense MVS gives $-1.4$ % (curry), $-20.8$ % (idli), $-69.4$ % (rice + dal) and $-82.9$ % (rice mound), a median of 45.1 %. Without texture on the food, the matcher has nothing to triangulate on its surface. Real rice and dal are textured, but smooth sauces and plain white foods are not, so this is the main limit of a passive-camera rig.

**Failure case.** `curry_heap__ceramic` is $-26.0$ %: its 6.2 mm plate noise raises the threshold above the low outer skirt of the heap, which is then cut from the footprint. The failure is mostly but not always repeated. Five runs of the identical input on this machine gave $-26.0$, $-26.0$, $-24.8$, $-23.1$ and $+0.2$ %, and one run on a second VM gave $+0.1$ %. OpenMVS is not deterministic on the textureless plate, and the noise estimate, and with it the threshold, moves between runs. The post-hoc hysteresis rule recovers it ($-1.0$ %) and lowers

TABLE III
E2 (NUTRITION5K). MASS PREDICTED AS $\rho V$ WITH A SINGLE DENSITY $\rho$ FITTED ON FOUR FOLDS AND SCORED ON THE HELD-OUT FOLD (5-FOLD CV). BRACKETS: 95 % BOOTSTRAP CI. †200-DISH SEEDED SUBSET.

| Volume source | $n$ | MAE (g) | median APE (%) |
|---|---|---|---|
| RealSense depth (sensor ceiling) | 503 | 139.3 [129.3, 150.0] | 68.2 [63.1, 72.9] |
| DepthPro†, anchor | 191 | 184.3 [159.0, 212.8] | 79.1 [73.3, 83.7] |
| DA-V2-B, oracle | 485 | 177.2 [164.4, 191.1] | 83.8 [82.0, 86.1] |
| DepthPro†, oracle | 191 | 199.1 [172.5, 228.6] | 84.4 [79.0, 87.1] |
| DA-V2-S, oracle | 483 | 178.1 [161.6, 196.6] | 85.7 [84.2, 87.5] |
| DA-V1-S, oracle | 491 | 180.4 [167.2, 194.1] | 88.2 [85.7, 89.6] |
| DA-V2-L, oracle | 486 | 185.3 [172.1, 199.0] | 89.0 [87.5, 90.1] |
| ZoeDepth-NYU, anchor | 487 | 186.8 [173.3, 201.0] | 89.1 [87.1, 91.5] |
| DA-V2-Met-S, anchor | 475 | 182.3 [168.4, 197.0] | 92.2 [89.7, 94.0] |
| Zoe-NYU-KITTI, anchor | 480 | 195.8 [181.8, 210.0] | 95.1 [94.4, 96.2] |
| ZoeDepth-NYU, oracle | 487 | 197.3 [184.0, 211.2] | 95.5 [94.9, 96.3] |
| DA-V2-L, anchor | 491 | 185.6 [171.0, 200.9] | 96.3 [95.4, 97.7] |
| MiDaS-DPT-Hybrid, oracle | 490 | 193.2 [178.7, 208.5] | 96.9 [95.9, 97.6] |
| DA-V2-Met-S, oracle | 475 | 195.2 [180.5, 211.5] | 96.9 [95.8, 97.6] |
| DA-V2-Met-B, oracle | 419 | 186.2 [171.0, 201.6] | 97.0 [95.2, 98.3] |
| Zoe-NYU-KITTI, oracle | 480 | 200.0 [186.2, 214.1] | 97.6 [97.2, 98.0] |
| DA-V2-Met-L, anchor | 473 | 198.4 [184.7, 212.8] | 97.8 [96.9, 98.4] |
| DA-V2-B, anchor | 492 | 186.9 [172.8, 202.0] | 97.9 [96.5, 98.8] |
| DA-V2-Met-L, oracle | 473 | 198.8 [185.1, 213.2] | 98.1 [97.3, 98.7] |
| DA-V2-S, anchor | 490 | 189.0 [175.8, 202.7] | 98.7 [97.9, 99.3] |
| DA-V2-Met-B, anchor | 421 | 192.8 [177.6, 208.4] | 98.8 [97.8, 99.3] |
| DA-V1-S, anchor | 487 | 201.6 [186.4, 218.5] | 99.5 [99.4, 99.7] |
| MiDaS-DPT-Hybrid, anchor | 441 | 194.0 [180.1, 208.4] | 99.9 [99.8, 100.0] |
| Predict the mean mass (no image) | 503 | 117.9 [109.8, 126.5] | 52.0 [46.1, 57.2] |

TABLE IV
E3 (SYNTHETIC, EXACT MESH VOLUME). ABSOLUTE VOLUME ERROR, MEDIAN AND 95% BOOTSTRAP CI OVER SCENES; BIAS IS THE MEAN SIGNED ERROR. TIME IS VM CPU SECONDS PER SCENE.

| Method | Subset | $n$ ok | $\|e\|$ % median [95% CI] | bias % | <10% | s |
|---|---|---|---|---|---|---|
| MVS dense (24 views) | all | 22/22 | 5.0 [2.3, 7.8] | -10.8 | 16 | 119 |
| MVS dense (24 views) | textured food | 16/16 | 4.6 [2.0, 6.1] | -4.3 | 13 | 120 |
| MVS dense (24 views) | ceramic | 8/8 | 5.9 [2.7, 26.0] | -5.9 | 5 | 117 |
| MVS dense (24 views) | steel | 8/8 | 3.0 [0.2, 5.1] | -2.7 | 8 | 123 |
| MVS dense (24 views) | banana leaf | 2/2 | 4.8 [1.9, 7.8] | +2.9 | 2 | 111 |
| MVS dense (24 views) | ceramic, textureless food | 4/4 | 45.1 [1.4, 82.9] | -43.6 | 1 | 113 |
| MVS dense, hysteresis (post-hoc) | all | 22/22 | 2.7 [1.1, 8.8] | -0.7 | 16 | 119 |
| MVS dense, hysteresis (post-hoc) | textured food | 16/16 | 2.2 [0.7, 5.0] | +5.5 | 13 | 120 |
| MVS dense, hysteresis (post-hoc) | ceramic | 8/8 | 5.5 [2.4, 24.2] | +11.5 | 5 | 117 |
| MVS dense, hysteresis (post-hoc) | steel | 8/8 | 0.6 [0.1, 2.0] | -0.4 | 8 | 123 |
| MVS dense, hysteresis (post-hoc) | banana leaf | 2/2 | 5.3 [1.9, 8.8] | +3.4 | 2 | 111 |
| MVS dense, hysteresis (post-hoc) | ceramic, textureless food | 4/4 | 25.6 [1.0, 58.4] | -27.7 | 1 | 113 |
| SfM sparse (24 views) | all | 22/22 | 91.3 [79.5, 99.5] | -81.4 | 2 | 18 |
| SfM sparse (24 views) | textured food | 16/16 | 84.5 [75.4, 94.8] | -75.3 | 2 | 20 |
| SfM sparse (24 views) | ceramic | 8/8 | 90.0 [70.2, 99.6] | -85.7 | 0 | 20 |
| SfM sparse (24 views) | steel | 8/8 | 78.9 [3.4, 99.3] | -65.0 | 2 | 21 |
| SfM sparse (24 views) | banana leaf | 2/2 | 93.2 [86.5, 100.0] | -93.2 | 0 | 16 |
| SfM sparse (24 views) | ceramic, textureless food | 4/4 | 100.0 [98.4, 100.0] | -99.6 | 0 | 15 |

the textured-food median error to 2.2 %, but its bias is $+5.5$ % and it overestimates the tall mound on ceramic by 46.1 %, lumpy rice by 24.2 % and the thali by 17.9 %, where plate noise connects to the food. No single height threshold works on every scene. We keep the pre-registered rule as primary and conclude that *separating food from plate*, not depth accuracy, is now the dominant error of the multi-view pipeline. Image-space food segmentation projected onto the height field is the natural next step. **Sparse is not enough.** Sparse SfM points alone have a median error of 91.3 % [79.5, 99.5] over all 22 scenes, nearly always an underestimate: feature points concentrate on the textured mat and food edges, leaving most of the food surface empty. Two steel scenes happen to land within 10 % (thali $-0.8$ %, tall mound $+3.4$ %), but the steel median is still 78.9 %. Densification is therefore required, and it is the main cost of a scan. Dense reconstruction takes about 119 s per scene, against 18 s for sparse on the VM CPU (VM evidence).

### D. E4: Ablations

The ablations change one factor at a time and use the dense MVS pipeline, except the marker-scale ablation, which used sparse SfM: number of views (6, 8, 12, 16, 24, evenly spaced), image resolution (1600×1200, 800×600, 640×480, the OV2640 UXGA, SVGA and VGA modes), JPEG re-encoding with OpenCV at quality 30 and 10 on its 0–100 scale (this is not the OV2640 0–63 quality register, and we did not measure how the two map), a deliberate error in the printed marker size ($\pm1$, $\pm2$ %), plate material, and three repeated runs on identical input to measure the run-to-run spread of SfM/MVS, which are not bit-deterministic. A run fails if SfM returns no model, if no marker side is triangulated, or if densification errors; failures are counted, not dropped. The driver (`experiments/synth/run_suite.py`) and the analysis script are in the repository. **Marker-scale error (60 runs, sparse SfM).** We scaled the believed marker side by 0.98–1.02 on 12 scenes. The recovered metric scale tracks the injected factor, but the integrated volume does not change monotonically (mean 71.7, 90.3, 77.9, 66.9 and 47.7 mL for factors 0.98 to 1.02). Two sparse runs of the same input differ by a median of 1.9 and up to 62.9 percentage points of true volume, so run-to-run SfM variation is far larger than the $\pm6$ % that the cube law predicts for a $\pm2$ % scale error. The cube law is therefore an analytic bound (Sec. VI-B), and this ablation is uninformative on sparse reconstructions. It will be repeated on dense reconstructions.

**View count (60 dense runs, 12 scenes).** Accuracy collapses below 12 views. The median absolute volume error is 91.6 % [78.7, 96.3] at 6 views and 71.4 % [22.7, 76.5] at 8 views (0 and 2 of 12 scenes within 10 %). It is 3.3 % [0.4, 19.2] at 12, 1.6 % [0.4, 3.6] at 16 and 4.2 % [0.4, 5.6] at 24 views (9, 11 and 12 of 12 within 10 %). The paired difference between 16 and 24 views is not significant ($p = 0.38$, Wilcoxon, $n = 12$). Every view registered at every setting; what drops is dense coverage (median 0.17 M points at 6 views against 1.03 M at 24). Four idli runs at 6 and 8 views return fewer than 1,000 food points and $-100$ % volume while reporting success, so the pipeline needs a minimum-food-point check before a scan is accepted. We keep 24 views as the default because it is the only setting with all scenes within 10 %; 16 views is the fallback when scan time matters. These runs were split across three identical cloud VMs (8-vCPU Xeon, same package versions). The pipeline is not bit-reproducible: the same 24-view input reconstructed on two machines differed by a median of 0.9 percentage points, 10 of 12 scenes within 1.2 points, but by 4.9 points on `thali3__ceramic` and 26.1 points on `curry_heap__ceramic` ($-26.0$ % vs. $+0.1$ %).

**Resolution and compression (72 dense runs, 12 scenes).** Resolution matters less than compression. Median absolute volume error is 3.9 % [0.8, 5.1] at 1600×1200 lossless and 3.2 % [2.1, 4.0] at 800×600 lossless (12 of 12 within 10 %;

TABLE V
E5: EATEN VOLUME FROM BEFORE/AFTER 24-VIEW DENSE MVS ON PARTLY EATEN SYNTHETIC PLATES (EXACT MESH VOLUMES). SHARE: ERROR IN THE EATEN FRACTION OF THE SERVING, PERCENTAGE POINTS.

| Plate | $n$ | Median \|err\| % [95 % CI] | $\leq$10 % | Share (pp) |
|---|---|---|---|---|
| All | 12 | 2.8 [2.0, 7.4] | 11/12 | 2.8 |
| Ceramic | 6 | 7.4 [2.2, 9.8] | 5/6 | 4.3 |
| Steel | 6 | 2.1 [1.5, 3.4] | 6/6 | 2.1 |

paired $p = 0.27$). OpenCV quality 30 at full resolution is 7.4 % [2.4, 8.6] (12 of 12 within 10 %, $p = 0.18$). Quality 10 fails at every resolution: 72.7 % at 1600×1200, 90.4 % at 800×600 and 82.7 % at 640×480 (at most 1 of 12 within 10 %; $p = 4.9 \times 10^{-4}$ for each against the lossless baseline, significant after Holm correction over five comparisons). Heavy compression removes the fine texture that dense matching needs. For the rig this means a low compression setting on the camera matters more than its maximum resolution; we did not measure how the OV2640 quality register maps onto these OpenCV levels, so the rig default has to be checked on hardware.

**Repeat runs (18 dense runs, 6 scenes, one machine).** Three runs on identical input stay within 0.2–0.9 percentage points of each other on steel plates and within 1.8–2.9 points on ceramic. The spread is again larger on the textureless plate. Two observations are already available from E3. Ceramic and steel differ in plate noise by about 2–5× (plate-material ablation). Two independent reconstructions of the same eight scenes differed by up to 2.7 percentage points (e.g. `idli_pair__ceramic` $-8.4$ vs $-5.7$ %), which bounds run-to-run repeatability before the formal repeat experiment.

### *E. E5: Partly eaten plates*

Each of the six plates from E3 was paired with a leftover version on ceramic and steel: one or more items removed, the rest reduced in height and radius (e.g. the rice eaten and 60 % of the dal height left), rendered with the same seed, camera path and plate. Eaten volume is the difference of two independent reconstructions, so the errors of both scans add. Across 12 pairs the median absolute error of the eaten volume is 2.8 % [2.0, 7.4], 11/12 within 10 %, and the eaten share of the serving is off by a median of 2.8 percentage points (Table V). The largest error is curry heap on ceramic (-11.1 %). The leftover scans alone are much worse in relative terms (median 9 % of the leftover volume), because thin remains on a plate are hard to reconstruct; in absolute terms that is a median 7.5 ml, small next to the served portion. The load cell remains the source of the grams; these numbers say how far the image-based cross-check can be trusted, and they are synthetic, not physical, measurements.

## VIII. LIMITATIONS

**Nutrition5k has no volume ground truth.** The dataset provides overhead RealSense depth and weighed mass, not food volume. We therefore score monocular models against the volume integrated from sensor depth over the same dish region (food plus plate rim above the support plane). This measures how well a model reproduces a depth sensor, not how well it measures food. Our attempt to separate food from plate with a second plate-floor plane fit gave a food volume that barely tracks weighed mass (Spearman $\rho = 0.25$, Pearson $r = -0.02$, $n = 503$), so we do not use it as a target. Absolute food-volume accuracy is reported only on synthetic scenes with exact mesh volume.

**Synthetic-to-real gap.** Blender scenes have perfect camera intrinsics, controlled lighting and procedural textures. Real OV2640 frames add rolling shutter, auto-exposure drift, lens distortion and JPEG blocking on low-texture food. The resolution and JPEG ablations bound part of this gap but not all of it. Textureless food already fails in simulation (Sec. VII-C); an active texture projector or image-space segmentation would be needed for it.

**No physical measurements yet.** Load-cell linearity, repeatability and drift, turntable angular error, scan duration and Wi-Fi transfer latency have scripts and empty tables (Sec. V); none of the numbers in this paper come from the physical rig. Timings are CPU timings on 8-vCPU cloud VMs, not on the ESP32-S3.

**Volume is not mass.** Converting volume to grams needs a per-food density. We do not estimate density; the load cell measures mass directly, which is the reason it is part of the rig.

**No clinical or user study.** The app, the alert rule and the weekly report have not been used by older adults, families or clinicians. Usability for people with poor eyesight or tremor, acceptance, alert fatigue and any effect on intake or outcomes are untested, and the residents in the app's demo data are fictional. A pilot in a ward or care home, with weighed plate waste as reference and staff and family feedback, is the next step.

## IX. CONCLUSION

AssistVision turns a plate scale and a turntable camera into a meal-intake monitor for older adults: the plate is weighed and scanned when the meal is served and again when it is cleared, the app logs grams eaten, family members are alerted after a run of low-intake meals, and the doctor gets a weekly report. The rig costs less than INR 5,000. The vision experiments explain the design. A single overhead image anchored only on the table does not recover food volume: the best deployable models (ZoeDepth-NYU, and Depth Pro at 15× the latency) still have median errors of 44–48 %, and low depth error (AbsRel) does not imply low volume error. Dense 24-view MVS with marker scale reaches a median error of a few percent on textured synthetic food and measures eaten volume on partly eaten plates to a median 2.8 %, but it fails on textureless food, and volume from depth does not predict grams on real mixed dishes, which is why grams come from the load cell. Next we will run the physical bench tests and real-plate captures, add image-space food segmentation, and

pilot the app with residents, families and staff in a care home, using weighed plate waste as the reference.

## Acknowledgment

Parts of the software and a first draft of this text were prepared with an AI coding assistant (Devin, Cognition AI). The authors checked, edited and take responsibility for all content, code and results.

## Reproducibility

All code, scene specifications, per-dish result files and the scripts that generate every table and figure are in the repository. Evidence categories are labelled throughout: *Nutrition5k* (public real RGB-D data), *synthetic* (Blender/Cycles renders with exact mesh volume), *VM* (timings on an 8-core CPU-only cloud VM, not on the ESP32), and *physical* (pending rig measurements).

## References

[1] M. Hiesmayr, K. Schindler, E. Pernicka, C. Schuh, A. Schoeniger-Hekele, P. Bauer, A. Laviano, A. D. Lovell, M. Mouhieddine, T. Schuetz, S. M. Schneider, P. Singer, C. Pichard, P. Howard, C. Jonkers, I. Grecu, and O. Ljungqvist, "Decreased food intake is a risk factor for mortality in hospitalised patients: The NutritionDay survey 2006," *Clinical Nutrition*, vol. 28, no. 5, pp. 484–491, 2009.

[2] D. Volkert, A. M. Beck, T. Cederholm, A. Cruz-Jentoft *et al.*, "ESPEN guideline on clinical nutrition and hydration in geriatrics," *Clinical Nutrition*, vol. 38, no. 1, pp. 10–47, 2019.

[3] A. J. Heighington-Wansbrough and L. Gemming, "Dietary intake in hospitals: A systematic literature review of the validity of the visual estimation method to assess food consumption and energy and protein intake," *Clinical Nutrition ESPEN*, vol. 52, pp. 296–316, 2022.

[4] Y. Kawasaki, M. Sakai, K. Nishimura, K. Fujiwara *et al.*, "Criterion validity of the visual estimation method for determining patients' meal intake in a community hospital," *Clinical Nutrition*, vol. 35, no. 6, pp. 1543–1549, 2016.

[5] Q. Thames, A. Karpur, W. Norris, F. Xia, L. Panait, T. Weyand, and J. Sim, "Nutrition5k: Towards automatic nutritional understanding of generic food," in *CVPR*, 2021, pp. 8903–8911.

[6] J. Dehais, M. Anthimopoulos, S. Shevchik, and S. Mougiakakou, "Two-view 3d reconstruction for food volume estimation," *IEEE Trans. Multimedia*, vol. 19, no. 5, pp. 1090–1099, 2017.

[7] F. P. W. Lo, Y. Sun, J. Qiu, and B. Lo, "Image-based food classification and volume estimation for dietary assessment: A review," *IEEE J. Biomed. Health Inform.*, vol. 24, no. 7, pp. 1926–1939, 2020.

[8] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE TPAMI*, vol. 44, no. 3, pp. 1623–1637, 2022.

[9] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, "Depth anything: Unleashing the power of large-scale unlabeled data," in *CVPR*, 2024.

[10] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, "Depth anything v2," in *NeurIPS*, 2024.

[11] S. F. Bhat, R. Birkl, D. Wofk, P. Wonka, and M. Müller, "Zoedepth: Zero-shot transfer by combining relative and metric depth," *arXiv:2302.12288*, 2023.

[12] A. Bochkovskii, A. Delaunoy, H. Germain, M. Santos, Y. Zhou, S. R. Richter, and V. Koltun, "Depth pro: Sharp monocular metric depth in less than a second," in *ICLR*, 2025.

[13] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *CVPR*, 2016.